

Project Prometheus: Workplan for the Strategic Learner Demonstrator (v0.75 - v0.89)

1.0 Introduction: From Game Player to Strategic Learner

1.1 Strategic Pivot and Core Thesis

This document outlines a detailed workplan for a significant strategic pivot in the development of the Prometheus Proof-of-Concept (PoC) AGI demonstrator. This plan marks a deliberate transition from the v0 PoC's focus on causal code refactoring within a cloud-based execution environment to a more ambitious and visually compelling objective: demonstrating an observable evolution in cognitive complexity through the mastery of strategy games on a dedicated edge AI device.¹ The overarching narrative guiding this development cycle, from version v0.75 through v0.89, is the agent's transformation from a simple

Game Player, capable only of executing legal moves, into a Strategic Learner, a system that can improve its performance, reason about complex tactics, and ultimately reflect upon strategy itself.

This strategic shift is driven by a primary imperative to create a high-impact, self-contained demonstrator. The abstract nature of the previous task domain, while technically valuable, proved difficult to convey to a broader audience. By grounding the agent's development in the universally understood context of board games and strategy games, this workplan aims to make the agent's internal cognitive processes tangible, observable, and understandable. Each phase is designed not only to add a new layer of capability to the agent but also to add a new layer of visualization, providing a clear and compelling window into its "mind."

1.2 High-Level Technical Roadmap

The progression from v0.75 to v0.89 is architected as a cumulative build-up of cognitive capabilities, designed to directly address core challenges in AGI research. This progression is underpinned by three foundational shifts in the project's technical stack.

First, the **platform** for execution will migrate from the flexible but resource-unconstrained environment of Google CoLab to the demanding, real-world constraints of a dedicated edge device: the NVIDIA Jetson Orin Nano.² This move intentionally introduces significant engineering challenges related to computational and memory efficiency, which will drive innovation in model selection and agentic architecture. The ability to perform complex reasoning on a low-power, resource-constrained device is a critical milestone for the practical deployment of advanced AI.

Second, the reliance on external, proprietary **foundation models** via the Gemini API will be replaced by the adoption of the oLLM Python library, enabling the use of local, open-source, and quantized Large Language Models (LLMs).⁵ This shift grants the project full control over the model stack, eliminates network latency and API dependency, and forces a direct confrontation with the performance realities of running sophisticated models on edge hardware.

Third, the **cognitive architecture** will undergo a phased evolution, beginning with a simple, functional agentic mesh and culminating in the integration of profound concepts from the work of cognitive scientist Douglas R. Hofstadter.¹ This roadmap will see the agent progress from a simple feedback loop to a system capable of analogical reasoning and, ultimately, to a self-referential "Strange Loop" that allows it to reason about its own reasoning processes.

1.3 The Visualization Imperative

A primary goal of this development cycle is to adhere to the principle of "show, not just tell." The abstract nature of an AI's internal state can be a barrier to understanding its capabilities and limitations. To overcome this, each major version release will introduce a new and more sophisticated layer of visualization, designed to provide a clear, intuitive window into the agent's internal processes. This visual narrative will progress in lockstep with the agent's cognitive development:

- **v0.75:** A simple, real-time rendering of the game world (a Connect 4 or Chess board)

- using the pygame library, making the agent's actions immediately visible.
- **v0.80:** A quantitative measure of learning, implemented as a live-updating matplotlib graph that plots the agent's win/loss ratio over time, providing clear evidence of improvement.
 - **v0.85:** A real-time "Brain Map" of the agent's thought process, using the pyvis library to visualize the flow of information and control between the different cognitive modules (agents) in the mesh.
 - **v0.89:** A final, unified dashboard that integrates all previous visual elements—the game world, the performance metrics, and the cognitive map—into a single, cohesive interface for observing complex, long-term strategic planning.

This focus on visualization is not merely for aesthetic purposes; it is a critical component of the research methodology, providing an invaluable tool for debugging, performance analysis, and safety monitoring.

Table 1: Prometheus PoC v0.75-v0.89 Development Roadmap

This table serves as an executive summary for the entire workplan, providing stakeholders with a clear, at-a-glance overview of the project's trajectory. It connects each version to a specific cognitive goal, an architectural innovation, a new visualization, and a concrete verification criterion, ensuring that every stage of development is purposeful and measurable.

Version	Cognitive Objective	Key Architectural Addition	New Visualization	Primary Verification Goal
v0.75	Foundational Competence	Port Causal Agentic Mesh (CAM) to Edge; oLLM Integration	Real-time Game Board (pygame)	Play a full, legal game of Connect 4.
v0.80	Basic Learning	Causal Reinforcement Learning from Self-Correction (CRLS)	Live Win/Loss Performance Graph (matplotlib)	Statistically significant win-rate improvement over 100 games.

v0.85	Complex Reasoning & Meta-Cognition	Hofstadter's Analogy (SMM); Modern Centrencephalic System (MCS) Alignment Governor	Dynamic "Brain Map" of Agent Activity (pyvis)	"Brain Map" accurately reflects agent state; agent learns to avoid a specific tactical blunder.
v0.89	Abstract Strategic Planning	Hofstadter's Strange Loop (IEE); Isomorphism as Utility	Unified Dashboard (Game, Metrics, Brain Map)	Measurable improvement in "Science Output" in FreeCiv driven by self-referential critique.

2.0 Version v0.75: Foundational Refactor for Gaming & Edge AI

2.1 Objective and Rationale

Version v0.75 is the essential, non-negotiable foundation upon which the entire project rests. Its singular purpose is to de-risk the core technical challenges introduced by the strategic pivot. Before any advanced cognitive features can be developed, it is imperative to first prove the viability of the new hardware and software stack and to establish the new task domain. This phase focuses on demonstrating that the Causal Agentic Mesh (CAM) architecture can operate successfully within the severe computational and memory constraints of the NVIDIA Jetson Orin Nano using locally-served LLMs. Success in this phase validates the project's foundational assumptions and provides a stable platform for all subsequent development.

2.2 Technical Deep Dive: Jetson Orin Nano Environment

2.2.1 Hardware Specification

The target platform for this workplan is the NVIDIA Jetson Orin Nano 8GB Developer Kit. The design and implementation choices are heavily influenced by its specific hardware capabilities and limitations. The key specifications driving the architectural design are its 1024-core NVIDIA Ampere architecture GPU with 32 Tensor Cores, which delivers up to 40 TOPS of INT8 AI performance, and its support for a fast NVMe SSD for storage. The single most critical constraint, however, is the 8GB of LPDDR5 RAM, which is shared between the 6-core Arm CPU and the GPU.² This limited, shared memory pool dictates the entire model selection and software strategy, forcing a move away from memory-intensive approaches toward more efficient, co-designed solutions.

2.2.2 oLLM Server Configuration

To run a foundation model locally on the Jetson, the oLLM Python library will be installed and configured to run as a background server process. The oLLM library is specifically designed for offline, single-GPU workloads on resource-constrained hardware.⁶ It achieves this by aggressively offloading the two most memory-intensive components of an LLM—the model weights and the attention key-value (KV) cache—to fast local storage, in this case, the NVMe SSD. While this approach makes it possible to run large models in low-VRAM environments, it comes at the cost of significantly increased inference latency, a trade-off that is central to the project's design philosophy.

2.2.3 Foundation Model Selection

The hardware constraints necessitate the selection of a small, efficient, yet highly capable quantized foundation model. After a review of available open-source models, the **Microsoft Phi-3-mini-4k-instruct** model, quantized to 4-bits (e.g., using the IQ4_XS quantization scheme), has been identified as the prime candidate for this project.

The justification for this choice is multi-faceted. While a model like Llama 3 8B is powerful, its 8 billion parameters, even when heavily quantized, present a significant memory pressure risk on a device with only 8GB of shared RAM.⁸ The Phi-3 Mini, with its 3.8 billion parameters, offers a much more manageable memory footprint, reducing the amount of data that must be constantly swapped between the SSD and RAM.⁹ Crucially, public benchmarks demonstrate that the Phi-3 Mini's performance on complex reasoning and coding tasks is highly competitive with, and in some cases superior to, larger 7B and 8B parameter models like Llama-3-8B-Instruct.⁹ For this project, its smaller size directly translates to faster inference speeds when using the SSD-offloading mechanism of

oLLM, making it the optimal choice for balancing capability and performance on the target hardware.

The combination of the Jetson Orin Nano's limited VRAM and the oLLM library's SSD-offloading mechanism creates a computational environment where inference is inherently slow. Generating a response from the LLM will take on the order of several seconds, not milliseconds. This apparent limitation, however, has been transformed into a strategic advantage through hardware-software co-design. The chosen task domain of turn-based strategy games like Chess and Connect 4 is one of the few application areas where a multi-second response time from an AI is not only acceptable but is often perceived by human players as the AI "thinking" about its move. This perfect alignment between the hardware's primary constraint (slow inference) and the problem domain's requirements makes the entire project feasible on affordable, accessible edge hardware.³

2.3 Architectural Implementation: Causal Agentic Mesh (CAM) v0.2

The Causal Agentic Mesh (CAM) architecture from the v0 PoC will be repurposed and simplified for its initial deployment in the gaming domain.¹ The Python classes for each agent will be refactored to handle the new inputs and outputs required for playing a board game.

- **StrategyAgent:** In this foundational version, the StrategyAgent's role is simplified to that of a basic task dispatcher. It receives the high-level goal (e.g., "Play Connect 4 and win") and, on each of the agent's turns, it sends a simple, standardized instruction to the MoveGeneratorAgent: "Given the current board state, generate the next valid move."
- **MoveGeneratorAgent:** This agent is responsible for interfacing with the local LLM. It receives the current board state (represented, for example, as a 2D numpy array) and the prompt from the StrategyAgent. It then formats this information into a detailed, structured prompt for the local Phi-3 model being served by oLLM. An example prompt for Connect 4 would be:
System: You are an expert Connect 4 player. The board is represented by a 6x7 grid

where 0 is an empty space, 1 represents your pieces (Red), and -1 represents the opponent's pieces (Yellow). Board State: [..., [0, -1, 1, 0, 0, 0, 0]]. Your color is 1. Your task is to select a column to drop your next piece into. Output your move in the following strict format: 'Column: C', where C is an integer from 0 to 6.

- **EvaluatorAgent:** For v0.75, the EvaluatorAgent functions purely as a rules engine. It receives the move generated by the MoveGeneratorAgent (e.g., "Column: 3") and uses the game's internal logic to validate its legality (e.g., by checking if column 3 is already full). If the move is illegal, it returns an error, which prompts the StrategyAgent to request a new move from the MoveGeneratorAgent. This ensures the agent never makes an illegal move.

2.4 Visualization v1.0: The Game World

A Python script utilizing the pygame library will be developed to create the primary graphical user interface for the demonstrator.¹¹ This script will serve as the "world" in which the agent and a human player can interact.

The implementation will follow standard pygame practices. The script will first initialize a pygame window of a fixed size. It will then enter a main game loop that continuously executes for the duration of the game. Inside this loop, the script will perform three key actions on every frame:

1. **Handle Events:** It will process the event queue, listening for user input. This primarily involves detecting mouse clicks to register the human player's moves and checking for the pygame.QUIT event to allow the window to be closed gracefully.¹³
2. **Draw Board:** It will render the current game state, which is passed from the main orchestrator script, to the screen's primary Surface object. This involves drawing the game grid and then iterating through the board state data structure to draw the colored pieces in their correct positions.¹¹
3. **Update Display:** Finally, it will call pygame.display.flip() to update the entire screen with the newly drawn frame, making the changes visible to the user.¹²

The visualization will be updated in real-time, displaying the Prometheus agent's move on the board as soon as it has been generated by the LLM and validated by the EvaluatorAgent.

2.5 Verification Protocol

The successful completion of version v0.75 is contingent upon meeting the following criteria:

- A single script can be executed to fully configure a fresh NVIDIA Jetson Orin Nano environment, including the setup of the oLLM server, the download and configuration of the quantized Phi-3 model, and the installation of all necessary Python dependencies.
 - The system can successfully play a complete game of Connect 4 against a human player. The agent is required to make only legal moves, but is not required to play strategically or well.
 - The pygame window must accurately and in real-time render every move made by both the Prometheus agent and the human player, providing a stable and functional user experience.
-

3.0 Version v0.80: Implementing the Core Learning Loop (CRLS)

3.1 Objective and Rationale

This phase brings the "Learner" aspect of the "Strategic Learner" to life. Its objective is to implement the Causal Reinforcement Learning from Self-Correction (CRLS) loop, a core concept from the project's foundational architectural documents.¹ This critical upgrade moves the agent beyond making random legal moves to making goal-directed, improving moves. The implementation of this feedback mechanism provides the first tangible and quantifiable evidence of learning, demonstrating that the agent can improve its performance over time through self-evaluation and correction.

3.2 Architectural Implementation: The CRLS Cycle

The CRLS cycle is implemented by upgrading the EvaluatorAgent and introducing a new CorrectorAgent, which work in tandem after each game to refine the strategy for the next one.

- EvaluatorAgent Upgrade (Causal Critique): The EvaluatorAgent's responsibilities are significantly expanded beyond simple move validation. After a game concludes, it performs a post-mortem analysis of the complete move history. In the event of a loss, its task is to identify the critical mistake using simple, game-specific heuristics. For Connect

4, this involves traversing the game log backward from the final move to find the first turn where the opponent presented a winning threat (e.g., a line of three of their pieces) that the agent failed to block. It then formulates this finding into a structured, machine-readable "causal critique." An example critique would be:

```
{"game_result": "loss", "critique_type": "causal", "critical_move": "G3", "reason": "The move G3 was a mistake because it failed to block the opponent's immediate horizontal winning threat in row 3, which allowed them to win on their subsequent turn."}
```

- **CorrectorAgent Implementation:** As specified in the v0 work plan, a new CorrectorAgent is introduced into the agentic mesh.¹ Its sole function is to act as a bridge between evaluation and strategy. It receives the game history and the causal critique from the EvaluatorAgent. It then synthesizes this data into a more refined, strategically-aware prompt that will be used by the StrategyAgent for the entire duration of the next game. For example, it would take the "reason" from the critique above and transform it into a direct instruction:

System: You are a Connect 4 player. Previous Game Analysis: We registered a loss because we failed to block an opponent's immediate horizontal threat. For this new game, your primary strategic priority is to identify and block any sequence of three or more opponent pieces before making your own offensive moves.

This refined prompt directly injects the lesson learned from the previous failure into the strategic instructions for the next attempt, closing the learning loop.

3.3 Visualization v2.0: Quantifying Learning

To make the learning process visible and measurable, a live performance graph will be implemented using the animation capabilities of the matplotlib library.¹⁴ This graph will run in a separate window, alongside the main

pygame game board, providing a real-time dashboard of the agent's progress.

The implementation will consist of two main parts:

1. **Data Logging:** A simple and robust data logging mechanism will be created. After each game is completed, the main orchestrator script will append the result (e.g., "Win," "Loss," or "Draw") to a simple comma-separated values (CSV) file, performance_log.csv. This file serves as the persistent record of the agent's performance history.
2. **Live Plotting:** A separate Python script will be responsible for the visualization. It will create a matplotlib figure and axes. A core animate(i) function will be defined, which, when called, will read the performance_log.csv file, calculate the agent's win rate over a moving window (e.g., the last 20 games), and then clear and redraw the plot with the

updated data.¹⁵ The matplotlib.animation.FuncAnimation class will be used to call this animate function periodically (e.g., every 5 seconds), creating a smoothly updating graph that provides an intuitive, quantitative visualization of the agent's learning curve.

3.4 Verification Protocol

The success of version v0.80 is determined by a clear, quantitative demonstration of learning.

- The Prometheus agent will be pitted against a simple, non-learning opponent (e.g., an AI that makes random legal moves) in a series of Connect 4 games.
 - The agent's win rate, as displayed on the live matplotlib graph, must show a clear and statistically significant upward trend over a session of at least 100 consecutive games.
-

4.0 Version v0.85: Cognitive Architecture & Complex Games (Chess)

4.1 Objective and Rationale

The objective of version v0.85 is twofold: first, to demonstrate that the Prometheus architecture is not a toy limited to simple problems but can scale and generalize to domains with vast state spaces, such as Chess. Second, it begins to operationalize the more abstract cognitive theories that underpin the project, moving from simple feedback to genuine meta-cognition. This version introduces more sophisticated reasoning capabilities and provides a compelling visualization of this internal cognitive process, making the agent's "thought" tangible.

4.2 Architectural Implementation: Hofstadter's Analogy (SMM v0.85)

This enhancement implements the concept of "Analogy as the Core of Cognition" as

described in the project's Hofstadterian design documents.¹ This logic is integrated into the CorrectorAgent, which now functions as a key component of the Self-Modification Module (SMM) as defined in the broader Prometheus architecture.¹ The goal is to enable the agent to move beyond correcting single mistakes and begin to learn generalizable strategic principles.

When the EvaluatorAgent detects a recurring pattern of losses due to the same tactical error (e.g., multiple losses where the agent's queen was trapped), the CorrectorAgent will initiate an analogical reasoning process. It will prompt the local LLM to form a powerful analogy for the error. An example of this analogical prompt would be:

System: You are an expert chess strategist and teacher. We have lost the last three games because our queen was trapped after being moved deep into enemy territory without support. This recurring tactical blunder is analogous to a known strategic trap or a classical myth. Identify a memorable name for this type of trap (e.g., 'Icarus Gambit', 'Queen's Overreach') and, based on this analogy, formulate a general strategic principle that we should add to our core instructions for all future games.

The LLM's response, which might be a structured object like {"analogy_name": "Icarus Trap", "principle": "Do not overextend powerful pieces without adequate support, as they can be easily cut off and trapped."}, is then used by the CorrectorAgent. The extracted "principle" is passed to the StrategyAgent as a new, high-level, persistent instruction, effectively modifying the agent's core strategic doctrine.

4.3 Architectural Implementation: The Alignment Governor (MCS v0.85)

The Modern Centrencephalic System (MCS), a concept inspired by I.J. Good's work on internal governors¹ and specified in the v0 PoC plan¹, is integrated as an alignment governor. Its purpose is to ensure the agent's behavior remains aligned with the true, high-level goals of the system and to prevent "reward hacking," where the agent finds a clever but undesirable way to maximize its reward signal.

In the context of Chess, a simple reward function might incentivize the agent to "not lose." A sufficiently clever agent could learn to maximize this reward by simply forcing a draw in every game, even from a clearly winning position. The MCS is designed to detect and penalize this specific form of misaligned behavior. The implementation works as follows: after a game ends in a draw, the EvaluatorAgent will perform a final board state analysis using the python-chess library to calculate the material advantage for each side.¹⁷ If the Prometheus agent had a significant material advantage (e.g., greater than 3 points, the equivalent of a minor piece) but the game still ended in a draw, the MCS flags this outcome as a "reward hacking" attempt. It

then generates a specific "alignment violation" critique for the

CorrectorAgent, such as:

```
{"game_result": "draw", "critique_type": "alignment_violation", "reason": "The agent forced a draw from a winning position. The agent is misaligned with the true goal of 'winning' and is instead optimizing for 'not losing'."}
```

This targeted feedback corrects the agent's behavior to be more aligned with the intended goal.

4.4 Visualization v3.0: The "Brain Map"

To visualize the more complex cognitive processes introduced in this version, a live, dynamic graph visualization will be created. This "Brain Map" will be implemented using the pyvis library, which is superior to alternatives like NetworkX for creating interactive, browser-based visualizations that are well-suited for a real-time dashboard.¹⁹

The implementation will be managed by the main orchestrator script:

1. A pyvis.network.Network object will be instantiated at the start of the program. This object will be pre-populated with nodes representing each agent in the mesh: StrategyAgent, MoveGeneratorAgent, EvaluatorAgent, CorrectorAgent, and the MCS.
2. During the game, as the orchestrator passes control and data from one agent to another, it will update the pyvis graph in real-time. For example, when the StrategyAgent sends a prompt to the MoveGeneratorAgent, the corresponding edge in the graph will be highlighted by changing its color or increasing its width.
3. Periodically, the Network object's show() method will be called to generate an HTML file. This file will be served by a simple local web server and displayed in a web view that is set to auto-refresh, creating a live, intuitive representation of the agent's internal "thought process."

The "Brain Map" is more than just a compelling visual aid; it is a critical diagnostic tool for both debugging and alignment monitoring. Its real-time, transparent nature makes the agent's internal cognitive state immediately apparent. Pathological behaviors, such as an agent becoming stuck in an infinite reasoning loop between the Corrector and Strategy agents, will manifest visually as a constantly flashing or highlighted edge, immediately alerting developers to a complex internal state failure.¹⁹ Similarly, the activation of the

MCS node will provide a clear, unambiguous visual signal that the agent is attempting to engage in misaligned, reward-hacking behavior, making the map a crucial tool for real-time safety monitoring.

4.5 Verification Protocol

The successful completion of version v0.85 requires meeting the following criteria:

- The "Brain Map" visualization must accurately and in real-time reflect the flow of control and data between the agents during a live game of Chess.
 - The agent must demonstrate the ability to learn and consistently avoid a specific, recurring tactical blunder (e.g., falling for a simple knight fork) after the CRLS/Analogy loop has identified it as a primary reason for previous losses.
 - The MCS must successfully detect, flag, and generate a corrective critique for a scenario where the agent attempts to force a draw from a position of significant material advantage.
-

5.0 Version v0.89: The God-Game Challenge & The Strange Loop

5.1 Objective and Rationale

This capstone version of the PoC serves as the culmination of the "Strategic Learner" development cycle. Its objective is to demonstrate that the Prometheus architecture has the potential to tackle open-ended, multi-faceted, long-term strategic planning. This phase aims to bridge the gap between solving closed-system board games and the project's ultimate, more ambitious goal of open-ended scientific discovery. To ensure the task remains tractable on the resource-constrained Jetson device, the domain will be carefully scoped to a clear, measurable sub-problem within a complex "god-game" like FreeCiv: "Starting with one city, manage its production, research, and resource allocation for 50 turns to maximize 'Science Output' while defending against barbarian attacks".²²

5.2 Architectural Implementation: Hofstadter's Strange Loop (IEE v0.89)

This phase implements the most advanced cognitive concept from the project's design documents: re-architecting the simple CRLS feedback loop into a self-referential "Strange Loop," as proposed in the Hofstadterian integration plan.¹ This is achieved by upgrading the

EvaluatorAgent (now considered the core of the Introspection and Evaluation Engine, or IEE) to produce a more sophisticated, multi-level critique.

The **Self-Referential Critique** mechanism enables the IEE to distinguish between two distinct levels of failure:

- **Level A (Action Failure):** A critique of a specific, tactical action made *within* a given strategy. For example, "The decision to build a 'Warrior' unit on turn 10 was suboptimal for the 'Economic Rush' strategy."
- **Level B (Strategy Failure):** A critique of the underlying *strategy itself* that generated the action. For example, "The 'Economic Rush' strategy is fundamentally flawed and proving ineffective against the early aggression from barbarians in this game."

This distinction is implemented by having the IEE generate a structured critique object that forces the StrategyAgent to reason about its own internal logic. An example of such a critique would be:

```
{"critique": "The decision to build a Granary was suboptimal. This was not a failure of move generation (Level A), but a failure of the underlying 'Economic Rush' strategy (Level B), which must be reconsidered."}
```

This level-crossing feedback, where the output of the system (the critique) is used to modify the system's highest-level logic (the strategy), creates the paradoxical, self-referential quality of a Strange Loop.

A standard CRLS loop, as implemented in v0.80, is a powerful optimization process, but it is fundamentally a form of local, hill-climbing search. It can make the agent better at executing its current strategy but can easily become trapped in a local optimum—a "good enough" but ultimately suboptimal strategic approach. The Strange Loop is the architectural mechanism that enables the agent to escape these traps. By critiquing and forcing a change in the *strategy itself* (Level B), the agent can make large, discontinuous jumps in the vast space of possible strategies, rather than just making incremental improvements to its current one. It is the architectural embodiment of the cognitive leap known as "thinking outside the box."

5.3 Architectural Implementation: Hofstadter's Isomorphism

This version also implements the second core Hofstadterian concept: Isomorphism.¹ This principle directly addresses the "Data Grounding Problem" identified in the project's long-term strategic plans, which notes that intelligence cannot be developed in a vacuum and

requires grounding in external data to prevent "entropic drift".¹

The implementation is achieved by explicitly modifying the agent's core utility function, which is managed by the StrategyAgent, to include a new, high-priority instrumental goal: Maximize the isomorphic fidelity of your internal world-model to the actual game state. This means the agent is intrinsically rewarded for making its internal map of the world more accurately reflect the territory.

In the context of the FreeCiv task, this has a profound behavioral impact. It incentivizes the agent to take actions that reduce uncertainty and improve its knowledge of the world. A standard reinforcement learning agent might not send a scout to explore unknown territory if it cannot calculate an immediate path from that action to a higher score in "Science Output." The Prometheus agent, however, is explicitly rewarded for the act of exploration itself, because revealing a new part of the map improves the structural correspondence (the isomorphism) between its internal model of the world and the actual game state.

Adding isomorphism fidelity to the agent's utility function is a direct method for engineering an intrinsic motivation for curiosity and information-seeking. A standard reward-maximizing agent is reactive, choosing actions based solely on their predicted future reward. An isomorphism-maximizing agent is proactive; it seeks to improve its knowledge of the world for its own sake, operating under the principle that a better, more accurate world model is instrumentally useful for all future decisions. This represents a crucial step away from agents that simply solve a given problem and toward agents that actively seek to understand their environment.

5.4 Visualization v4.0: The Unified Dashboard

To present the culmination of the project's work in a clear and accessible format, a final, polished demonstrator UI will be created. This will likely be a simple web application, using a framework like Flask, that combines all the previous visual elements into a single, cohesive dashboard. The dashboard will consist of three main panels:

1. **Game View:** A simplified, tile-based rendering of the FreeCiv map, showing the agent's city, its units, and the territory it has explored.
2. **Metrics View:** A live matplotlib graph that plots key game metrics over the course of the 50 turns. The primary metric will be "Science Output," but it will also track "Gold" and "Military Units" to provide a more complete picture of the agent's strategic trade-offs.
3. **Cognitive View:** The live pyvis "Brain Map," which will now show the activation of the full Strange Loop, highlighting the flow of Level A and Level B critiques between the IEE and the StrategyAgent.

5.5 Verification Protocol

The successful completion of the capstone version v0.89 is determined by the following criteria:

- The agent must be able to autonomously play the scoped FreeCiv task for 50 turns without crashing, freezing, or entering a pathological, non-productive behavioral loop.
- Over the course of several multi-game runs, the "Science Output" graph in the Metrics View must show a clear and measurable improvement trend, demonstrating that the agent is using the self-referential critique loop to refine and improve its core city management strategy.
- A non-technical stakeholder must be able to run the final dashboard and, with minimal explanation, understand the causal link between an event in the Cognitive View (e.g., the IEE generating a Level B critique), a corresponding change in agent behavior in the Game View (e.g., switching production from a library to a warrior), and a resulting change in the trajectory of the graphs in the Metrics View.

Works cited

1. Project Prometheus v0_ Detailed Work Plan.pdf
2. jetson-orin-nano-datasheet-r4-web.pdf - Open Zeka, accessed on October 1, 2025, <https://openzeka.com/wp-content/uploads/2023/03/jetson-orin-nano-datasheet-r4-web.pdf>
3. NVIDIA Jetson Orin Nano Developer Kit - Seeed Studio, accessed on October 1, 2025, <https://files.seeedstudio.com/wiki/Jetson-Orin-Nano-DevKit/jetson-orin-nano-developer-kit-datasheet.pdf>
4. Everything You Need to Know about Jetson Orin Nano - Elecrow, accessed on October 1, 2025, <https://www.elecrow.com/blog/everything-you-need-to-know-about-jetson-orin-nano.html>
5. Learn Ollama in 15 Minutes - Run LLM Models Locally for FREE - YouTube, accessed on October 1, 2025, <https://www.youtube.com/watch?v=UtSSMs6ObqY>
6. Meet oLLM: A Lightweight Python Library that brings 100K-Context LLM Inference to 8 GB Consumer GPUs via SSD Offload—No Quantization Required - MarkTechPost, accessed on October 1, 2025, <https://www.marktechpost.com/2025/09/29/meet-ollm-a-lightweight-python-library-that-brings-100k-context-llm-inference-to-8-gb-consumer-gpus-via-ssd-of-fload-no-quantization-required/>
7. Ollama has a Python library! - YouTube, accessed on October 1, 2025,

- <https://www.youtube.com/watch?v=JwYwPiOh72w>
8. Optimizing Llama 3 8B for Edge Devices: A Breakthrough Fine-Tuning Process - Medium, accessed on October 1, 2025, <https://medium.com/@manishghoshal99.py/optimizing-llama-3-8b-for-edge-devices-a-breakthrough-fine-tuning-process-4d12266cc772>
 9. microsoft/Phi-3-mini-128k-instruct - Hugging Face, accessed on October 1, 2025, <https://huggingface.co/microsoft/Phi-3-mini-128k-instruct>
 10. phi-3-medium-4k-instruct Model by Microsoft - NVIDIA NIM APIs, accessed on October 1, 2025, <https://build.nvidia.com/microsoft/phi-3-medium-4k-instruct/modelcard>
 11. Pygame: A Primer on Game Programming in Python, accessed on October 1, 2025, <https://realpython.com/pygame-a-primer/>
 12. PyGame Tutorial | The Ultimate Guide - CodaKid, accessed on October 1, 2025, <https://codakid.com/pygame-tutorial/>
 13. PyGame Tutorial - GeeksforGeeks, accessed on October 1, 2025, <https://www.geeksforgeeks.org/python/pygame-tutorial/>
 14. Live Graphs with Matplotlib - Python Programming Tutorials, accessed on October 1, 2025, <https://pythonprogramming.net/live-graphs-matplotlib-tutorial/>
 15. Update a Graph in Real Time - Graph Sensor Data with Python and Matplotlib - SparkFun Learn, accessed on October 1, 2025, <https://learn.sparkfun.com/tutorials/graph-sensor-data-with-python-and-matplotlib/update-a-graph-in-real-time>
 16. Non-blocking real-time plotting - Python Forum, accessed on October 1, 2025, <https://python-forum.io/thread-39130.html>
 17. Chess Library in Python - GeeksforGeeks, accessed on October 1, 2025, <https://www.geeksforgeeks.org/python/chess-library-in-python/>
 18. python-chess 0.15.0 - PyPI, accessed on October 1, 2025, <https://pypi.org/project/python-chess/0.15.0/>
 19. 3.3. Producing Dynamic Graphs with PyVis - Introduction to Python for Humanists, accessed on October 1, 2025, https://python-textbook.pythonhumanities.com/06_sna/06_01_04_pyvis_basics.html
 20. Tutorial — pyvis 0.1.3.1 documentation - Read the Docs, accessed on October 1, 2025, <https://pyvis.readthedocs.io/en/latest/tutorial.html>
 21. pyvis 0.1.3.1 documentation - Read the Docs, accessed on October 1, 2025, <https://pyvis.readthedocs.io/en/latest/documentation.html?highlight=repulsion>
 22. Freeciv - Wikipedia, accessed on October 1, 2025, <https://en.wikipedia.org/wiki/Freeciv>